

Assignment 2 Report

Vinaykumar Reddy Junuthula

System Architecture

The experiments were conducted on the Lonestar6 supercomputer. Each compute node consists of AMD EPYC 7763 processors with 64 cores per socket and two sockets per node, resulting in 128 cores per node. The architecture is x86_64 with two NUMA nodes per system. Each core has L1 and L2 caches, and a large shared L3 cache is available across cores. The system provides high memory bandwidth and supports large-scale parallel execution using MPI.

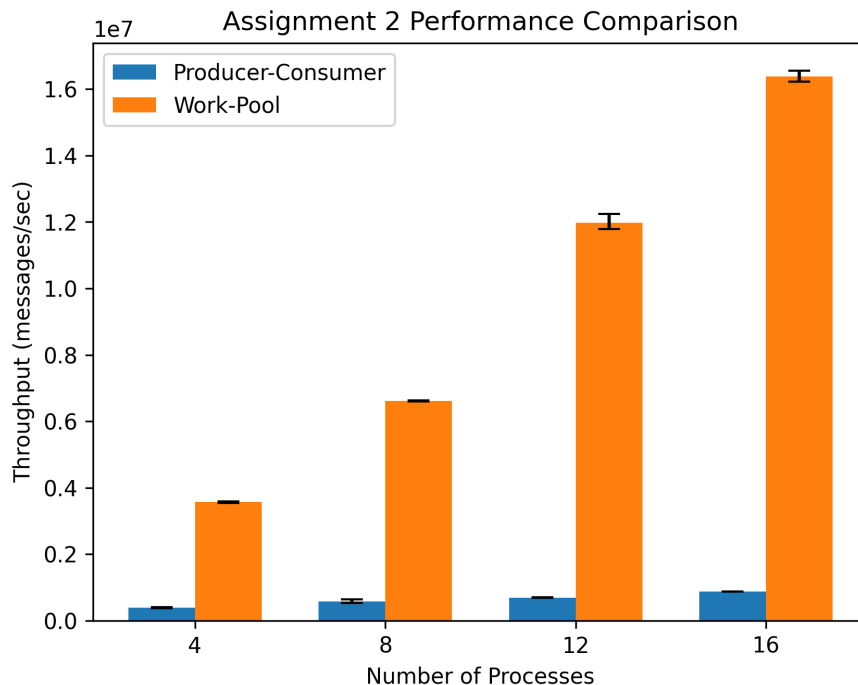
Experiment Setup

Two execution models were implemented using MPI: a producer-consumer model and a work-pool model. In the producer-consumer model, rank 0 acts as a centralized broker managing a bounded buffer. Producers send work messages to the broker, while consumers request work from it.

In the work-pool model, all processes act as both producers and consumers. Each process sends work messages to random processes and consumes incoming work directly, eliminating centralized coordination.

Each program was executed with 4, 8, 12, and 16 processes. For each configuration, five runs were performed for 120 seconds. Throughput was computed as the number of messages consumed per second. The average throughput is shown in the figure below, with error bars representing minimum and maximum values.

Performance Results



Observations

The results show that throughput increases with the number of processes for both execution models. However, the work-pool model significantly outperforms the producer-consumer model at all process counts.

The producer-consumer model suffers from a centralized bottleneck at the broker (rank 0), which limits scalability as the number of processes increases. This leads to reduced throughput and inefficient utilization of resources.

In contrast, the work-pool model distributes communication across all processes, eliminating centralized coordination overhead. This allows better parallelism and significantly higher throughput.

The error bars indicate variability across runs due to communication overhead, system scheduling, and runtime differences. Despite this variability, the trend clearly shows that decentralized communication provides superior performance.